

# مستند جامع: روش‌های بای‌پس کپچا و راهکارهای امن‌سازی آن

## مقدمه

**کپچا (CAPTCHA)** که مخفف "Completely Automated Public Turing test to tell Computers and Humans Apart" است، به عنوان چالشی برای تشخیص کاربر انسان از ربات طراحی شده است <sup>1</sup>. وبسایت‌ها برای جلوگیری از فعالیت‌های خودکار مخرب (مانند اسپم، حملات بروت فورس، سوءاستفاده از فرم‌ها و...) از انواع کپچا استفاده می‌کنند. ایده کپچا این است که حل آن برای انسان ساده باشد اما برای اسکریپت‌ها یا هوش مصنوعی دشوار <sup>2</sup>. با این حال، مهاجمان و ربات‌های پیشرفته راه‌های متعددی برای دور زدن یا بای‌پس کردن کپچا پیدا کرده‌اند. این مستند به سه بخش تقسیم می‌شود: ابتدا **تمام روش‌های شناخته‌شده بای‌پس کپچا** را بررسی می‌کنیم، سپس **راهکارهای ایمن‌سازی کپچا** در برابر این روش‌های حمله را توضیح می‌دهیم، و در پایان به **اصول پیاده‌سازی کپچای کاملاً امن** (به نحوی که حتی المقدور قابل‌بای‌پس نباشد) می‌پردازیم.

## روش‌های دور زدن (بای‌پس) انواع کپچا

مهاجمان باتجربه از طیف وسیعی از تکنیک‌ها برای دور زدن کپچاها استفاده می‌کنند. این روش‌ها هم شامل سوءاستفاده از ضعف‌های پیاده‌سازی در کپچاهای ساده‌تر است و هم بهره‌گیری از فن‌آوری‌های پیشرفته برای حل خودکار کپچاهای پیچیده. در ادامه، مهم‌ترین راه‌های بای‌پس کپچا را مرور می‌کنیم:

- **عدم اعتبارسنجی در سمت سرور (ضعف پیاده‌سازی):** یکی از رایج‌ترین آسیب‌پذیری‌ها این است که وبسایت صرفاً وجود فیلد کپچا را در سمت کلاینت بررسی می‌کند، اما پاسخ کپچا را در سمت سرور صحت‌سنجی نمی‌کند <sup>3</sup>. در چنین حالتی مهاجم می‌تواند به سادگی پارامتر مربوط به کپچا را از درخواست حذف کند یا مقدار دلخواه بدهد و اگر سرویس هیچ بررسی‌ای انجام ندهد، درخواست پردازش می‌شود <sup>4</sup>. حتی در برخی موارد، برنامه فقط حضور فیلد کپچا یا طول رشته را چک می‌کند و محتوای آن را نمی‌سنجد؛ نتیجه اینکه ارسال مقدار خالی ("") یا تغییر جزئی در مقدار کپچا ممکن است کافی باشد تا مکانیزم کپچا دور زده شود <sup>5</sup>. این یک باگ منطقی است که در برنامه‌های وب مشاهده شده و گاهی در رقابت‌های کشف باگ پاداش بالایی هم داشته است <sup>6</sup>. همچنین استفاده از کلیدها یا پاسخ‌های ثابت (مثلاً استفاده تصادفی از **Site Key** و **Secret Key** آزمایشی گوگل reCAPTCHA در حالت Production) می‌تواند باعث شود هر مقداری به عنوان پاسخ پذیرفته شود که شکستی فاجعه‌بار برای کپچا است.

- **تغییر روش HTTP یا قالب داده ارسالی:** برخی پیاده‌سازی‌های کپچا فقط برای درخواست‌های نوع خاص (مثلاً فقط POST) فعال هستند. اگر توسعه‌دهنده اعتبارسنجی کپچا را به درخواست POST محدود کرده باشد، تغییر متد درخواست به GET و حذف پارامتر کپچا می‌تواند منجر به بای‌پس شود <sup>7</sup>. به طور مشابه، تغییر نوع محتوا (Content-Type) می‌تواند موثر باشد؛ برای مثال اگر برنامه انتظار دریافت کپچا در بدنه JSON داشته باشد، اما مهاجم همان داده را به صورت `application/x-www-form-urlencoded` یا متن ساده بفرستد، ممکن است اعتبارسنجی کپچا دور بخورد <sup>8</sup>. این ترفندها در موارد متعددی کپچا را دور زده‌اند، چون سرویس در همه حالت‌ها اعتبارسنجی را اعمال نکرده است <sup>9</sup>. پس مهاجم با تغییر روش (مثلاً POST→GET یا POST→PUT) <sup>10</sup> و تغییر قالب ارسال، می‌تواند مکانیزم کپچا را غافلگیر کند.

- **استفاده مجدد از پاسخ کپچا (قابلیت Reuse):** در برخی از پیاده‌سازی‌ها، پاسخ کپچا یا توکن تأیید آن بیش از یک‌بار قابل استفاده است. اگر کپچا پس از یک بار حل شدن در سمت سرور ابطال نشود، مهاجم می‌تواند همان پاسخ یا توکن را برای چندین درخواست دیگر ارسال کند و همه را بگذراند <sup>11</sup> . برای مثال، دیده شده که یک پاسخ کپچا تا زمانی که جلسه اصلی در پروکسی باز نگه داشته شود، بارها قابل استفاده است <sup>12</sup> . این ضعف به ربات اجازه می‌دهد با یک بار حل کردن کپچا (یا حدس زدن آن) حملات پرشماری انجام دهد. بنابراین عدم مکانیزم "یک‌بار-مصرف" بودن کپچا نوعی آسیب‌پذیری است.

- **جا زدن خود به جای کاربر دیگر با هدرهای ساختگی:** برخی وبسایت‌ها مکانیزم کپچا یا نرخ درخواست را بر اساس آدرس IP کاربر مدیریت می‌کنند. در چنین مواردی، مهاجم می‌تواند هدرهای HTTP جعلی مانند X-Forwarded-For یا X-Remote-IP را به درخواست اضافه کند تا IP خود را مخفی یا تغییر دهد و سیستم را فریب دهد <sup>13</sup> . اگر برنامه بدون اعتبارسنجی، این هدرها را به عنوان IP کلاینت قبول کند، ربات می‌تواند با هر درخواست IP تازه‌ای جا بزند و محدودیت‌های کپچا/ریتم درخواست را دور بزند <sup>14</sup> . حتی گزارش شده ارسال دو بار هدر X-Forwarded-For با مقدار 127.0.0.1 توانسته کپچا/Rate Limit را دور بزند <sup>15</sup> . این روش مبتنی بر ضعف اعتماد سرویس به هدرهای کاربر است.

- **دور زدن با حفظ نشست و کوکی‌های اعتبار (Token Hijacking):** پس از حل شدن کپچا معمولاً توکن یا کوکی‌ای در مرورگر کاربر ذخیره می‌شود تا نشان دهد کاربر انسان است. مهاجمان می‌توانند این کوکی یا توکن را استخراج کرده و در اسکریپت خود استفاده کنند تا دیگر کپچا نمایش داده نشود <sup>16</sup> <sup>17</sup> . برای مثال، کپچای Cloudflare Turnstile در صورت موفقیت کوکی cf\_clearance را ست می‌کند که تا مدتی اعتبار دارد <sup>16</sup> . ابزارهایی مانند Camoufox حتی به ربات اجازه می‌دهند یک مرورگر واقعی باز کرده و کپچا را یک‌بار به صورت دستی حل کند و سپس کوکی حل‌شده را ذخیره نموده و در درخواست‌های خودکار بعدی ارسال کند <sup>18</sup> . این کار عملاً یک بلیت عبور برای ربات فراهم می‌کند که بدون نیاز به حل دوباره، به صفحات محافظت‌شده دسترسی یابد.

- **شبیه‌سازی رفتار انسانی و دور زدن تشخیص‌ها:** بسیاری از سیستم‌های ضدربات (مانند reCAPTCHA v3 یا Turnstile) علاوه بر کپچا، رفتار کاربر (الگوی حرکت ماوس، سرعت کلیک، زمان‌بندی درخواست‌ها و غیره) را تحلیل می‌کنند <sup>19</sup> . ربات‌های پیشرفته برای دور زدن این تشخیص‌ها، رفتار خود را انسان‌نما می‌کنند؛ مثلاً بین درخواست‌ها تأخیر تصادفی می‌گذارند، از کلیک‌ها و حرکت‌های ماوس شبیه‌سازی‌شده استفاده می‌کنند و از انجام تعداد زیادی درخواست در زمان بسیار کوتاه خودداری می‌کنند <sup>20</sup> <sup>21</sup> . علاوه بر این، آن‌ها User-Agent خود را مرتباً تغییر می‌دهند تا شبیه مرورگرهای مختلف شوند <sup>22</sup> و از IP‌های متنوع (با کمک شبکه‌های پروکسی یا VPN) بهره می‌برند تا از یک آدرس واحد درخواست انفجاری نفرستند <sup>23</sup> . این تکنیک‌ها باعث می‌شود مکانیزم‌های کپچا به سختی بتوانند الگوی ثابتی برای شناسایی ربات پیدا کنند. به عبارت دیگر، ربات با **استتار هویت و رفتار** خود، سعی می‌کند کلاً کپچا ظاهر نشود یا کمتر ظاهر شود. در حملات اسکریپتی به وبسرویس‌ها، گردش تصادفی بین چندین IP و تغییر امضای درخواست (Headers, Cookies) یک روش متداول برای پرهیز از مواجه شدن مکرر با کپچا است <sup>23</sup> <sup>24</sup> .

- **حل خودکار کپچا با هوش مصنوعی (الگوریتم‌های یادگیری ماشین):** امروزه پیشرفت هوش مصنوعی باعث شده بسیاری از کپچاهای تصویری و متنی دیگر "غیرقابل حل" نباشند. الگوریتم‌های OCR (تشخیص نویسه نوری) می‌توانند متن‌های تحریف‌شده در کپچاهای قدیمی را تشخیص دهند <sup>25</sup> . شبکه‌های عصبی بینایی قادرند اشیاء خواسته‌شده را در کپچاهای تصویر-محور شناسایی کنند (مثلاً تشخیص عکس‌هایی که در آن‌ها چراغ راهنمایی وجود دارد) <sup>26</sup> . پژوهش سال 2023 نشان داد هوش مصنوعی می‌تواند کپچاهای تصویری را با دقت 96٪ حل کند، در حالی که نرخ موفقیت انسان‌ها بین 50٪ تا 85٪ است <sup>27</sup> <sup>28</sup> ! حتی کپچاهای صوتی (مثلاً حروف خوانده‌شده برای کاربران نابینا) نیز با سرویس‌های گفتاربه‌متن قابل حل هستند <sup>29</sup> . برای نمونه، ابزارهایی مانند unCaptcha2 توانسته‌اند کپچای صوتی گوگل (reCAPTCHA Audio) را با حدود 85٪ دقت بشکنند (با استفاده از API گفتاربه‌متن خود گوگل) <sup>29</sup> . این آمار تکان‌دهنده نشان می‌دهد که در جنگ بین کپچا و هوش مصنوعی، فعلاً کفه ترازو به نفع هوش مصنوعی سنگین شده است.

- **بهره‌گیری از سرویس‌های حل کپچا توسط انسان:** هنگامی که حل الگوریتمی کپچا دشوار باشد، مهاجمان به سادگی از **نیروی انسانی ارزان** استفاده می‌کنند. وب‌سرویس‌ها یا API‌هایی وجود دارند (مانند 2Captcha, Anti-Captcha, DeathByCaptcha و...) که کپچاهای ارسالی شما را دریافت کرده و به یک حل‌کننده انسانی آنلاین می‌سپارند و پاسخ را برمی‌گردانند <sup>30</sup> <sup>31</sup>. این سرویس‌ها با استفاده از شبکه‌ای از انسان‌ها (اغلب کارگران آنلاین با دستمزد پایین) حتی پیچیده‌ترین کپچاها - اعم از تصاویر پیچیده، پازل‌ها و... - را حل می‌کنند <sup>32</sup> <sup>33</sup>. دقت حل توسط انسان بسیار بالا است و عملاً هر کپچایی که یک انسان قادر به حل آن باشد با این روش قابل گذر است <sup>34</sup>. تنها محدودیت این روش، **هزینه و زمان** است؛ حل هر کپچا چند ثانیه تا ده‌ها ثانیه طول می‌کشد و برای تعداد بسیار زیاد، هزینه دلاری (هر کپچا چند سنت) خواهد داشت <sup>35</sup>. با این وجود، برای مهاجمانی که به صورت انبوه اسپم ارسال می‌کنند یا ثبت‌نام‌های جعلی انجام می‌دهند، این هزینه ناچیز محسوب می‌شود. بنابراین "مزارع کپچا" (Captcha Farms) یا سرویس‌های حل کپچا امروزه از رایج‌ترین راه‌های دور زدن کپچا هستند که ماهیتی کاملاً **انسان‌محور** دارند و تشخیص آن‌ها از کاربر واقعی دشوار است <sup>36</sup>.

**جمع‌بندی این بخش:** همان‌طور که مشاهده شد، روش‌های بای‌پس کپچا از اشکالات ساده برنامه‌نویسی (مانند عدم بررسی سمت سرور یا وابستگی به ورودی‌های کنترل‌نشده) گرفته تا سوءاستفاده از ضعف‌های طراحی (مانند امکان استفاده مجدد یا روش‌های جایگزین درخواست) و حتی راهکارهای پیشرفته فنی (هوش مصنوعی و سرویس‌های انسانی) را در برمی‌گیرد. به همین دلیل تأمین امنیت کامل تنها با تکیه بر یک کپچا ثابت، روزبه‌روز دشوارتر می‌شود. در بخش بعد خواهیم دید که چگونه می‌توان مکانیزم‌های کپچا را در برابر این تهدیدات مقاوم‌تر کرد.

## راهکارهای امن‌سازی کپچا و جلوگیری از بای‌پس شدن

برای ایمن کردن کپچا در برابر روش‌های دور زدن فوق، نیاز به رویکردی چندلایه و توجه دقیق به جزئیات پیاده‌سازی است. در ادامه، راهکارها و **بهترین اقدامات (Best Practices)** جهت امن‌سازی انواع کپچا را مرور می‌کنیم. این توصیه‌ها به گونه‌ای هستند که حتی‌الامکان جلوی تکنیک‌های بای‌پس شناخته‌شده را بگیرند:

- **اعتبارسنجی کامل در سمت سرور:** اولین اصل، **هرگز به اعتبارسنجی سمت کاربر اکتفا نکنید**. پاسخ کپچا باید حتماً در سمت سرور و ترجیحاً با یک کلید یا سرویس مخفی بررسی شود. برای کپچاهای متنی ساده، پاسخ صحیح باید در سرور (مثلاً در سشن کاربر) ذخیره و مقایسه شود. برای سرویس‌هایی مانند reCAPTCHA یا hCaptcha نیز، توکن دریافت‌شده از کاربر باید به API پشتیبانی سرویس (Google, Cloudflare و...) ارسال و اعتبارسنجی گردد <sup>19</sup>. اطمینان حاصل کنید که بدون اجرای این بررسی سمت سرور، فرآیند موردنظر (مثلاً ثبت فرم یا ورود) کامل نشود. این کار جلوی اکثر موارد حذف پارامتر کپچا یا دستکاری آن در کلاینت را می‌گیرد. همچنین از کلیدهای خصوصی/عمومی مربوط به سرویس کپچا به‌درستی استفاده کنید و هرگز کلید مخفی را در کد جلوی کاربر قرار ندهید.

- **ایمن‌سازی مسیرهای کد و ورودی‌ها:** تمامی مسیرهای دسترسی به عملکرد حساس باید محافظت‌شده باشند. بررسی کنید که کپچا بر روی **تمامی متدهای HTTP** مربوط به آن عمل اعمال شود. اگر فقط برای POST طراحی شده، مطمئن شوید درخواست‌های GET یا سایر متدها قادر به انجام آن عمل نباشند (مثلاً با خطا یا ریدایرکت مواجه شوند) <sup>10</sup>. همین‌طور ورودی کپچا را فارغ از قالب داده (JSON، فرم‌دیتا و...) اعتبارسنجی کنید یا نوع داده ثابت مورد انتظار را مشخص کنید تا مهاجم نتواند با تغییر Content-Type آن را دور بزند <sup>8</sup>. در واقع، logic اعتبارسنجی کپچا باید در لایه سرور و قبل از انجام هر عملیاتی صورت گیرد و به سادگی با **تنظیمات غیرمعمول درخواست** قابل چشم‌پوشی نباشد.

- **جلوگیری از استفاده مجدد توکن یا پاسخ:** هر پاسخ کپچا باید یک‌بار مصرف باشد. اگر خودتان کپچا طراحی می‌کنید، پس از یک بار پاسخ صحیح باید آن پاسخ یا جلسه حاوی آن را باطل کنید. در کپچاهای متداول این موضوع رعایت شده (مثلاً توکن reCAPTCHA در صورت استفاده و اعتبارسنجی، دیگر اعتبار ندارد). اما اگر کپچایی طراحی کرده‌اید که پاسخ ثابت در روز دارد یا به ازای هر کاربر یک کپچای ثابت می‌دهد، حتماً مکانیسمی بگذارید

که پاسخ دوبار پذیرفته نشود. همچنین به زمان اعتبار پاسخ‌ها دقت کنید؛ پاسخ کپچا نباید برای مدت طولانی معتبر بماند. مثلاً کپچاهای متنی ساده را می‌توان به session ID کاربر گره زد تا فقط همان کاربر و تا زمانی کوتاه معتبر باشد. این اقدامات مانع از آن می‌شود که مهاجم یک پاسخ را ذخیره کرده و بارها استفاده کند

11 .

#### • عدم اعتماد به ورودی‌های کنترل‌نشده (مانند هدرهای سفارشی): هدرهایی نظیر

X-Forwarded-For یا سایر اطلاعاتی که کاربر می‌تواند تزریق کند، نباید بدون بررسی در منطق امنیتی نقش داشته باشند. اگر اپلیکیشن شما برای اعمال محدودیت کپچا بر IP کاربر تکیه می‌کند، تنها به IP اتصال TCP/SSL دریافتی اعتماد کنید یا از هدرهای سفارشی فقط در صورتی استفاده کنید که از سمت یک پروکسی مطمئن تنظیم شده باشند. مهاجم نباید بتواند با ارسال هدر جعلی، سیستم را گول بزند که گویا از IP دیگری آمده است<sup>14</sup>. در صورت امکان، اعتبار هدر X-Forwarded-For را با لیست سفید یا قالب صحیح بررسی کنید (مثلاً مقدار غیرمجاز 127.0.0.1 یا فرمت غیرعادی را نپذیرید). خلاصه اینکه، هر داده‌ای که می‌تواند توسط کلاینت دستکاری شود نباید باعث دور زدن محدودیت کپچا یا نرخ درخواست شود<sup>37</sup>.

• محدودسازی نرخ و تشخیص الگوهای حمله: کپچا آخرین سد دفاعی نیست! حتماً برای درخواست‌های حساس (مثلاً لاگین، ثبت‌نام، بازیابی رمز) محدودیت نرخ (Rate Limiting) نیز پیاده‌سازی کنید. اگر کاربری (یا IP مشخص) مثلاً ده بار پی‌پی‌کپچای اشتباه وارد کرد، می‌توان مشکوک شد که ربات است و برای مدت کوتاهی درخواست‌های بیشتر از او را بلاک کرد. همچنین اگر کاربری کپچا را بیش از حد سریع و مکرراً حل می‌کند، این می‌تواند علامت استفاده از اسکریپت یا سرویس خودکار باشد. در واقع ترکیب تحلیل رفتار و نرخ می‌تواند جلو برخی ربات‌ها را بگیرد<sup>38</sup>. برای مثال، اگر معمولاً انسان‌ها ۱۰~ ثانیه زمان صرف حل یک کپچا می‌کنند اما یک کلاینت همیشه در کمتر از ۱ ثانیه پاسخ می‌دهد، باید مکانیزم امنیتی شما آن را به عنوان رفتار غیرعادی پرچم‌گذاری کند. در چنین مواردی می‌توان اقدامات مضاعف انجام داد: مثلاً نمایش کپچای اضافه (دو مرحله پشت سرهم)، یا افزایش پیچیدگی کپچا برای آن کلاینت خاص.

• استفاده از سرویس‌ها و کپچاهای پیشرفته‌تر: امن‌ترین راهکار شاید استفاده از کپچاهای نسل جدید باشد که خود را با بات‌ها تطبیق می‌دهند. برای نمونه، Google reCAPTCHA v3 به طور نامرئی رفتار کاربر را امتیازدهی می‌کند و فقط در موارد مشکوک کاربر را به چالش (مثلاً reCAPTCHA v2) می‌کشانند<sup>39</sup> 40. یا سرویس Cloudflare Turnstile به کلی بدون چالش و به صورت پشت‌صحنه، کاربر را اعتبارسنجی می‌کند و با بهره‌گیری از یادگیری ماشین و سیگنال‌های متعدد تشخیص می‌دهد که ربات هست یا خیر<sup>41</sup>. این سرویس‌ها برای کاربر واقعی بسیار راحت‌ترند و برای ربات‌ها سخت‌تر (چون ده‌ها عامل از قبیل تاریخچه مرورگر، حرکت ماوس، زمان‌بندی کلیک، کانفیگ مرورگر، و ... تحلیل می‌شود)<sup>19</sup>. البته حتی این سیستم‌ها هم بی‌نقص نیستند و گزارش شده که در برابر بات‌های بسیار پیشرفته چالش دارند<sup>40</sup> 42. اما بدون شک استفاده از نسخه‌های به‌روز کپچا (مانند hCaptcha، Arkose Labs FunCaptcha، GeeTest و FriendlyCaptcha) امنیت را نسبت به کپچاهای قدیمی افزایش می‌دهد<sup>43</sup> 44. این خدمات مدرن، تکنیک‌های ضدهات را سریع‌تر به‌روزرسانی می‌کنند و در برابر حملات شناخته‌شده پچ می‌شوند.

• تصادفی‌سازی و تنوع در چالش‌ها: یکی از نقاط ضعف کپچاهای سنتی، الگوی ثابت و قابل پیش‌بینی آن‌هاست. برای مثال، کپچای متنی ممکن است همیشه از یک فونت و تحریف یکسان استفاده کند یا بانک محدودی از کلمات داشته باشد. ربات‌ها می‌توانند با جمع‌آوری نمونه‌ها و استفاده از یادگیری ماشین روی آن الگو، مدل حل ایجاد کنند<sup>45</sup>. راهکار دفاعی این است که کپچا را تا حد امکان تصادفی و متغیر کنید. اگر کپچا متنی طراحی می‌کنید، از فونت‌های متنوع، اعوجاج‌های مختلف، پس‌زمینه‌های شلوغ و نویزدار استفاده کنید به طوری که هیچ دو تصویری کاملاً مشابه نباشد. یا در کپچای تصویری، تصاویر پرسشی را مرتب عوض کنید و از تصاویر عمومی (که شاید در دیتاست‌های AI موجود باشند) کمتر بهره ببرید. ایده این است که مهاجم برای هر نمونه جدید مجبور باشد دوباره مدل‌سازی کند. البته این کار تا حدی تعارض با تجربه کاربری دارد (کپچای بسیار ناخوana برای انسان‌ها آزاردهنده است) و باید تعادل را رعایت کرد.

• **ترکیب چند نوع چالش (مولتی‌مدال):** در صورت امکان، از **چند لایه کپچا** برای افزایش امنیت استفاده کنید. برای مثال، برخی سرویس‌ها اگر به پاسخ کاربر شک کنند، مرحله دوم کپچا را نمایش می‌دهند. یا می‌توانید ترکیبی از یک سؤال ساده و یک پازل تصویری داشته باشید. ایده کلی این است که حل خودکار یا حتی انسانی کپچا را برای ربات‌سازها پرهزینه‌تر کنید. مثلاً **Arkose Labs** گزارش کرده که استفاده از پازل‌های تعاملی چندمرحله‌ای (مانند چرخاندن یک شیء سه‌بعدی تا جهت صحیح) باعث می‌شود اسکریپت‌های معمول نتوانند به سادگی عبور کنند و حتی استفاده از سرویس‌های انسانی نیز کندتر شود <sup>46</sup>. هرچند افزایش مراحل نباید بی‌دلیل باشد، اما به عنوان **step-up challenge** در صورت مشکوک بودن رفتار کاربر، می‌تواند موثر باشد.

• **بکارگیری تله‌ها و فیلترهای نامرئی:** یک روش خلاقانه برای به دام انداختن ربات‌های ساده‌تر، استفاده از **honeypot** است. Honeypot فیلدی مخفی در فرم است که برای کاربران انسانی قابل مشاهده یا پرکردن نیست (مثلاً با CSS مخفی می‌شود)، اما ربات‌های ساده که تمام فیلدها را پر می‌کنند ممکن است آن را پر کنند <sup>47</sup>. اگر دیدید این فیلد مخفی پر شده، می‌توانید آن تعامل را ربات فرض کرده و حتی قبل از رسیدن به کپچا رد کنید. این روش جلو بسیاری از ربات‌های سطح پایین را می‌گیرد و فشار را از روی مکانیزم کپچای اصلی کم می‌کند. همچنین می‌توانید از ورودی‌های چک‌باکس نامرئی "من ربات هستم" (!) استفاده کنید تا ربات‌های بدون هوش را گرفتار کنید. البته بات‌های پیشرفته‌تر با دیدن المان‌های مخفی‌شده متوجه این تله‌ها می‌شوند <sup>48</sup>، اما به هر حال داشتن چند لایه دفاعی بهتر از تکیه بر یک لایه است.

• **توجه به دسترس‌پذیری و تجربه کاربری در عین امنیت:** امن‌سازی کپچا نباید به قیمت قفل‌شدن در سایت به روی کاربران واقعی تمام شود. حتماً راهکارهای جایگزین برای کاربران دارای معلولیت فراهم کنید. برای مثال، کپچای صوتی برای افراد نابینا ضروری است؛ اما می‌دانیم که کپچای صوتی نسبتاً توسط ربات‌ها قابل حل است <sup>29</sup>. در اینجا شاید راهکار بهتر استفاده از **روش‌های تأیید هویت جایگزین** باشد، مثلاً به جای کپچای صوتی، یک **معما یا سؤال ساده شناختی** پرسیده شود که کاربر نابینا با صفحه‌خوان خود قادر به پاسخگویی باشد. همچنین همواره سطح سختی کپچا را با توجه به نرخ حملات تنظیم کنید. لازم نیست همه کاربران را با سخت‌ترین کپچا مواجه کنید؛ بسیاری از سرویس‌های مدرن رفتار مرحله‌ای دارند (ابتدا چک نامرئی، سپس کپچای ساده، و اگر مشکوک بود کاربر، کپچای سخت). این باعث می‌شود کاربران کمتری دل‌زده شوند و زمانی که کپچا واقعاً نمایش داده می‌شود بدانند که **ضرورت امنیتی** وجود داشته است. به طور خلاصه، **امنیت و کاربرپذیری** را با هم در نظر بگیرید؛ یک کپچای افراطی که هیچکس نتواند حل کند، سایت شما را نیز غیرقابل استفاده خواهد کرد.

با اجرای موارد فوق، می‌توان مقاومت کپچاها را در برابر بسیاری از تکنیک‌های بای‌پس شناخته‌شده افزایش داد. البته همچنان ممکن است ربات‌های بسیار پیشرفته بتوانند برخی از این موانع را پشت‌سر بگذارند یا ترکیبی از روش‌ها را به‌کار گیرند. از این رو در بخش بعد به این می‌پردازیم که **آیا می‌توان کپچای طراحی کرد که "کاملاً امن" باشد؟** و ویژگی‌های چنین کپچایی چیست.

## طراحی کپچای کاملاً امن - امکان‌پذیر یا رؤیایی؟

سوالی که مطرح می‌شود این است: **آیا می‌توان کپچایی پیاده‌سازی کرد که هیچ روشی قادر به بای‌پس آن نباشد؟** واقعیت این است که با پیشرفت مداوم هوش مصنوعی، هیچ کپچایی به طور مطلق شکست‌ناپذیر نخواهد بود <sup>49 50</sup>. با این حال، می‌توان اصولی را برای طراحی "ایمن‌ترین کپچای ممکن" بیان کرد که شکستن آن را به حداقل احتمال و حداکثر هزینه برای مهاجم برساند. در ادامه، ویژگی‌ها و توصیه‌های طراحی چنین کپچایی را بررسی می‌کنیم:

1. **استفاده از هوش مصنوعی و تحلیل رفتاری در قلب کپچا:** کپچای ایده‌آل صرفاً یک چالش ثابت نیست، بلکه یک **سیستم تشخیص بات پویا** است. این سیستم باید بر مبنای **یادگیری ماشین** رفتار کاربران را مدل کند و بتواند به صورت آنی (Real-Time) رفتارهای مشکوک را تشخیص دهد <sup>51</sup>. به عنوان نمونه، کپچای ایدئال می‌تواند در پس‌زمینه به داده‌های کاربر (مانند حرکت ماوس، الگوی کلیک، زمان توقف روی صفحه، مشخصات

مرورگر و سیستم) نمره ریسک بدهد و فقط وقتی این نمره از حد معینی بدتر بود، کاربر را به حل چالش وادار کند.<sup>39</sup> این رویکرد در reCAPTCHA v3 و سرویس‌هایی نظیر Cloudflare Turnstile دیده می‌شود که تلاش می‌کنند تا حد امکان **نامرئی** عمل کنند و فقط ربات‌ها را به دام بیندازند.<sup>41</sup> هوش مصنوعی مدافع همچنین می‌تواند خود را با حملات جدید تطبیق دهد؛ مثلاً اگر ربات‌ها شروع به تقلید از الگوی انسانی کنند، الگوریتم مدافع باید به سیگنال‌های عمیق‌تر (superficial نیستند) سوئیچ کند یا ترکیب سیگنال‌ها را تغییر دهد.<sup>52</sup> به بیان دیگر، یک کپچای امن در سال ۲۰۲۵ باید تا حدی از **AI در برابر AI** بهره ببرد.

2. **چالش‌های پویا، چندلایه و غیرقابل پیش‌بینی:** در طراحی کپچا باید فرض کنیم مهاجم بسیاری از جزئیات چالش ما را می‌داند (یا با مهندسی معکوس به دست خواهد آورد). پس بهترین دفاع، **تصادفی‌سازی و تنوع مداوم** است. کپچایی امن خواهد بود که هر بار تجربه‌ای کمی متفاوت ارائه دهد و نشود برای آن اسکریپت ثابت نوشت. به عنوان مثال، کپچای Arkose Labs با نام FunCaptcha ترکیبی از پازل‌های کوچک سه‌بعدی، تشخیص اشیاء و حتی بازی‌های کوتاه ارائه می‌دهد که بین کاربران متفاوت است.<sup>53</sup> این تنوع و پیچیدگی باعث می‌شود نوشتن ربات برای حل خودکار آن بسیار دشوار و پرهزینه شود. طراحی کپچای خودتان نیز باید از این ایده الهام بگیرد: مثلاً اگر سؤال امنیتی می‌پرسید، متن سؤال و ترتیب جواب‌ها و... را تصادفی کنید. اگر تصویری نشان می‌دهید، از میان یک مجموعه بسیار بزرگ و نامعمول انتخاب شود که قابل حدس نباشد. همچنین **چندمرحله‌ای بودن** می‌تواند مفید باشد؛ یعنی پس از گذر از مرحله اول، در صورت مشکوک بودن شرایط، مرحله دوم یا سوم داده شود (مرحله اضافی برای رباتی که با زحمت مرحله اول را رد کرده می‌تواند کمرشکن باشد). البته زیاده‌روی در مراحل برای همه کاربران توصیه نمی‌شود، بلکه باید **Adaptive** باشد: سیستم تنها وقتی احساس کند هنوز احتمال ربات بودن هست، چالش اضافه‌تر مطرح کند.<sup>55</sup> این همان رویکرد Adaptive Challenge است که برخی پلتفرم‌ها پیاده کرده‌اند.

3. **تکیه بر عوامل خارج از کنترل ربات‌ها:** هرچه چالش به **دنیای فیزیکی یا هویت انسانی** نزدیک‌تر شود، دور زدن آن سخت‌تر است. مثلاً کپچایی را در نظر بگیرید که از کاربر می‌خواهد با دوربین خود یک تصویر زنده بفرستد یا یک کار فیزیکی انجام دهد. بدیهی است که این نوع کپچاها را به سادگی نمی‌توان خودکار حل کرد (اگرچه مشکلات حریم خصوصی و تجربه کاربری دارند). نمونه عملی‌تر، استفاده از **بیومتریک** است: مثلاً اثر انگشت یا تشخیص چهره برای اطمینان از انسان بودن (گرچه این نیز قابل جعل و دارای ملاحظات است). ایده دیگر، تلفیق کپچا با **احراز هویت دومرحله‌ای** است: به عنوان مثال، پس از یک سری فعالیت ریسکی، کاربر را وادار کنیم شماره موبایلش را تأیید کند یا کدی از ایمیلش وارد کند. واضح است که این روش‌ها عملاً فراتر از کپچاهای سنتی هستند، اما اگر صحبت از «هیچ جوره نتوان دور زد» باشد، دخیل کردن مؤلفه‌هایی که نیاز به انسان واقعی و دارای هویت دارد بسیار موثر خواهد بود. با این حال، به دلیل پیچیدگی و هزینه معمولاً این روش‌ها را برای همه کاربران استفاده نمی‌کنند، بلکه در سناریوهای حساس یا در مواجهه با حملات سازمان‌یافته به کار می‌روند.

4. **استفاده از اطلاعات زمینه (Context) و هوشمندی بیشتر در طراحی پرسش:** کپچای امن می‌تواند پرسش‌هایی بپرسد که پاسخ آنها به دانش و زمینه انسانی نیاز دارد و در عین حال برای ماشین سخت است. برای مثال، پرسش‌های چالشی مانند «در این تصویر احساس افراد را تشخیص دهید» یا «کدام یک از این جملات مفهوم طنز دارد؟» شاید برای هوش مصنوعی امروز دشوارتر از یک مسأله ریاضی ساده باشد. البته با پیشرفت **مدل‌های زبانی و بینایی** شاید این زمینه‌ها هم ناامن شوند، اما طراحی خلاقانه پرسش‌ها (مثلاً ترکیب فهم زبانی و بینایی، یا پرسش از تجربیات عمومی انسان که ربات‌ها ندارند) می‌تواند کار را سخت‌تر کند. برخی کپچاهای نوین تلاش کرده‌اند با اضافه کردن **عامل زمان** این کار را کنند: مثلاً از کاربر می‌خواهند ۵ ثانیه روی یک دکمه کلیک کند یا ژست خاصی را برای چند لحظه نگه دارد. اینگونه تعاملات برای انسان بدیهی و برای ربات پیچیده‌تر است. خلاصه اینکه **در طراحی کپچا از خلاقیت انسانی بهره ببرید** و الزاماً به الگوهای تکراری قدیمی اکتفا نکنید.

5. **پایش و به‌روزرسانی مستمر مکانیزم کپچا:** فرض کنیم امروز کپچایی ساخته‌اید که هیچ رباتی نتواند آن را حل کند. چنین وضعیتی پایدار نخواهد ماند. مهاجمان ممکن است با **یادگیری ماشینی خصمانه** (Adversarial

(ML) الگوریتم شما را دور بزنند؛ یعنی حجم بزرگی داده از کپچاهای شما و پاسخشان جمع‌آوری و یک مدل حل اختصاصی برایش تربیت کنند <sup>56</sup> . بنابراین تیم امنیتی شما باید همواره آمار عملکرد کپچا را زیر نظر داشته باشد: نرخ موفقیت کاربران واقعی در مقابل نرخ عبور ربات‌ها. هرگاه شواهدی دیدید که ربات‌ها در حال وفق یافتن هستند (مثلاً ناگهان حجم زیادی حل موفق کپچا بدون افزایش معنادار کاربران واقعی رخ داد)، باید در مکانیزم خود تغییر ایجاد کنید. این تغییر می‌تواند در نوع سوالات، بانک تصاویر، سختی چالش یا پارامترهای تشخیص رفتار باشد. سرویس‌های بزرگ کپچا دقیقاً چنین می‌کنند؛ به طور مرتب نسخه‌های جدید ارائه می‌دهند (reCAPTCHA v1 → v2 → v3 و اخیراً Turnstile) تا از هوش مصنوعی جلوتر بمانند <sup>55</sup> . در طراحی داخلی هم اگر از هوش مصنوعی برای تشخیص استفاده می‌کنید، مدل‌های خود را با داده‌های جدید و حملات جدید retrain کنید. به عبارتی، **جنگ کپچا و بات یک فرایند پویا است** و شما هم باید پویا دفاع کنید.

6. **توازن امنیت با راحتی و حریم خصوصی:** در نهایت، "امن‌ترین کپچا" در خلأ معنی ندارد؛ مهم امنیت واقعی در بستر کاربردی است. هرچه کپچا را سخت‌تر کنیم، امکان اذیت شدن کاربران واقعی بیشتر می‌شود و حتی ممکن است برخی از خدمات شما صرف‌نظر کنند. همچنین برخی راهکارهای امنیتی (مثل تحلیل رفتار گسترده یا استفاده از داده‌های شخصی) نگرانی‌های **حریم خصوصی** ایجاد می‌کند <sup>57</sup> . برای نمونه، reCAPTCHA گوگل متهم است که داده‌های فراوانی از کاربر جمع‌آوری می‌کند و ممکن است با قوانین GDPR سازگار نباشد <sup>58</sup> . کپچای ایدئال باید به این ملاحظات هم توجه کند: یعنی **تا حد امکان امنیت را با حفظ حریم خصوصی و رضایت کاربر تأمین کند** . شاید به همین دلیل است که امروزه صحبت از جایگزین‌های کپچا می‌شود (مانند **استاندارد Privacy Pass** که Google و Cloudflare روی آن کار می‌کنند تا کاربر با یک توکن تأییدشده ناشناس به سایت‌های مختلف ثابت کند ربات نیست). هر چه باشد، هدف نهایی ایجاد **اعتماد** میان سرویس و کاربر واقعی، و جدا کردن عوامل مخرب، با کمترین تأثیر منفی است <sup>59</sup> . پس در طراحی کپچای بسیار امن، همیشه این سوال را از خود بپرسید که آیا راهکاری که پیشنهاد می‌دهم **هم برای کاربر خوب و هم برای بات بد قابل قبول است یا خیر** . راه‌حلهایی موفق خواهند بود که امنیت، تجربه کاربری و ملاحظات اخلاقی/قانونی را با هم متعادل کنند <sup>59</sup> .

**جمع‌بندی نهایی:** هیچ کپچایی را نمی‌توان تضمین‌شده "غیرقابل‌بای‌پس" دانست، اما با ترکیبی از **مکانیزم‌های هوشمند سمت سرور، چالش‌های چندلایه پویا، تحلیل رفتار کاربر، استفاده از خدمات و داده‌های به‌روز، و پایش مداوم** می‌توان به سطحی از امنیت دست یافت که بای‌پس کردن آن برای مهاجمان به شدت سخت، زمان‌بر و پرهزینه شود. روند حمله/دفاع در این حوزه نشان می‌دهد که با پیچیده‌تر شدن بات‌ها، کپچاها نیز باید تکامل پیدا کنند <sup>55</sup> . در مقام یک متخصص امنیت، توصیه می‌شود به جای تکیه بر یک کپچای ساده و عدم تغییر آن، همواره از جدیدترین فناوری‌های ضوابط بهره گرفته، تنظیمات کپچا را بر اساس رفتارهای مشکوک تنظیم کنید و آماده ارتقاء یا تعویض مکانیزم کپچا در مواجهه با نسل جدید حملات باشید <sup>60</sup> <sup>61</sup> . تنها با این رویکرد پویاست که می‌توان امیدوار بود کپچا همچنان به عنوان یک سد مؤثر در برابر ربات‌های مهاجم باقی بماند.

## منابع و مراجع:

1. December 18, 2025, *ZenRows Blog* ", Yuvraj Chandra, "7 Ways to Bypass CAPTCHA While Web Scraping" <sup>23</sup> <sup>2</sup> .
2. harshbothra\_, 31 Jan@*Twitter Thread* ", Harsh Bothra, "#learn365 Day-31: Captcha Bypass Techniques" <sup>62</sup> <sup>3</sup> 2021 .
3. Sep 2, 2020, *Medium* ", Akshat Singhal, "CAPTCHA Bypass Techniques" <sup>13</sup> <sup>10</sup> .
4. *Cobalt Pentest Wiki* ", Captcha Bypass - X-Forwarded-For" <sup>63</sup> .
5. Nov 18, 2019, *Codementor.io Article* ", Harshit Tyagi, "Solving CAPTCHA with Web Automation" <sup>64</sup> <sup>25</sup> .
6. *HackTricks GitBook* ", HackTricks - Captcha Bypass" <sup>29</sup> <sup>24</sup> .
7. April 10, 2024, *BBC Science Focus* ", Dr. Peter Bentley, "How AI finally won its war on CAPTCHA images" <sup>19</sup> <sup>27</sup> .

Feb 8, 2025 ,*CyberPeace Blog* "Aditi Pangotra, "Who Is Winning the War with AI: Bots vs. Captcha .8  
 Nov 15, 2024 ,*GeeTest Blog* ",Best CAPTCHA Services for Enhanced Cybersecurity in 2025 6" .9  
 Sep ,*Skyvern Blog* ",Suchintan Singh, "Best Way to Bypass CAPTCHA for AI Browser Automation (2025) .10  
 2025 ,08

- Ways to Bypass CAPTCHA While Web Scraping: Quick Guide 7 48 47 30 23 22 21 20 18 17 16 2 1  
 ZenRows  
<https://www.zenrows.com/blog/bypass-captcha-web-scraping>

Thread by @harshbothra\_ on Thread Reader App – Thread Reader App 62 36 12 11 7 6 5 4 3  
<https://threadreaderapp.com/thread/1355857615532646403.html>

CAPTCHA BYPASS TECHNIQUES !. Q.what is captcha???? according to... | by Akshat 15 14 13 10 9 8  
 singhal | Medium  
<https://honeyakshat999.medium.com/captcha-bypass-techniques-f768521516b2>

How AI finally won its war on CAPTCHA images | BBC Science Focus Magazine 52 27 19  
<https://www.sciencefocus.com/future-technology/ai-vs-captcha>

Captcha Bypass | hacktricks 29 24  
<https://angelica.gitbook.io/hacktricks/pentesting-web/captcha-bypass>

Solving CAPTCHA with Web automation | Codementor 64 31 25  
<https://www.codementor.io/@harshittyagi/solving-captcha-with-web-automation-10v3rnvr9>

?Who Is Winning the War with AI: Bots vs. Captcha 65 61 60 59 58 57 56 55 50 49 45 38 28 26  
<https://cyberpeace.org/resources/blogs/who-is-winning-the-war-with-ai-bots-vs-captcha>

CAPTCHA Bypass Methods for Browser Automation 2025 66 35 34 33 32  
<https://www.skyvern.com/blog/best-way-to-bypass-captcha-for-ai-browser-automation-september-2025>

Captcha Bypass | Pentest Vulnerability Wiki 63 37  
<https://www.cobalt.io/vulnerability-wiki/v2-authentication/captcha-bypass-x-forwarded-for>

Best CAPTCHA Services for Enhanced Cybersecurity in 2025 6 54 53 51 46 44 43 42 41 40 39  
<https://www.geetest.com/en/article/choosing-best-captcha-great-titles>